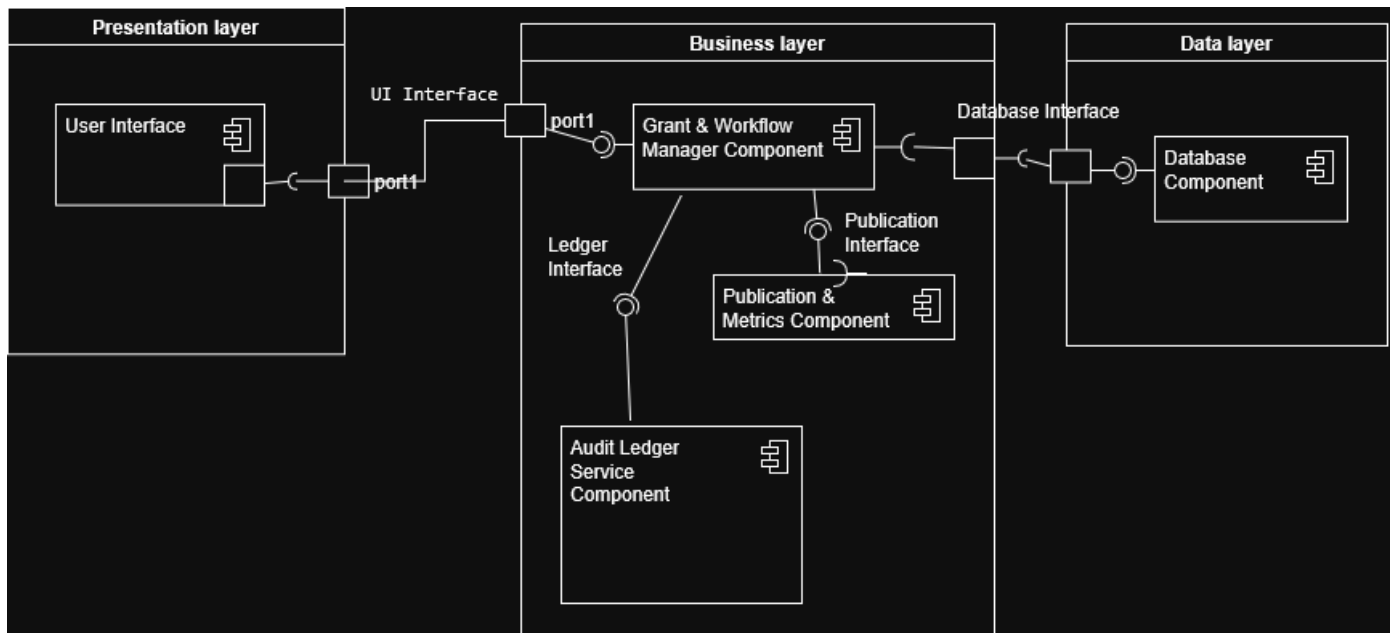


LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Faculty Research Grant & Publication Tracker



Architecture Selection

The **Layered Architecture** is the best fit for the Faculty Research Grant & Publication Tracker.

Reasons:

1. Simple Domain Model & Low System Boundary Friction

The requirements center on traditional CRUD operations (logging metrics, recording procurement, managing approval workflows) and reporting analytics. The system domain does not have separate sub-domains that scale at drastically different rates or require independent deployment pipelines. Layered architecture provides a clear separation of concerns (Presentation for UI/Dashboards, Business/Logic for approval rules and validations, and Data for storage) without the network overhead and operational bloat of microservices.

2. High Consistency & Transactional Integrity

Requirements **NFR-1** (Immutable audit ledger) and **NFR-2** (Expenses must never exceed granted funds) strictly demand strong consistency (*ACID* transactions).

- **Why Layered fits:** Enforcing NFR-2 requires transactional locks across funds and expenses. A Layered monolithic architecture with a unified database easily supports relational integrity, constraints, and atomic transactions.
- **Why Microservices struggles here:** Microservices rely on eventual consistency and distributed sagas, making strict multi-entity invariants like "prevent expenditure exceeding fund limits across services" complex and error-prone.

Security Advantage

Centralized Authorization & Enforcement at the Business Layer Boundary

In a **Layered Architecture**, all incoming API requests and data mutations must pass through a single, well-defined Business/Logic layer before reaching the Data layer. This strict horizontal boundary allows security mechanisms to be enforced uniformly in one place:

- **Role-Based Access Control (RBAC):** The Business layer evaluates access rules on every request (e.g., verifying that *only* users with the Research Dean role can call the Approve Grant Allocation endpoint, or that *only* the Primary Author can approve co-author credit).
- **Immutability Enforcement (NFR-1):** By keeping database access encapsulated within the Data layer behind business logic APIs, application users cannot execute direct SQL queries. The Data Access Objects (DAOs) can restrict queries to strictly append-only operations (INSERT) for audit tables, preventing unauthorized modifications or tampering with financial logs.

Performance Benefit

In-Memory Caching & Unified Database Queries (Meeting Sub-60-Second SLAs)

While the user requirement likely implies sub-second or real-time query responses (well under 60 seconds), a Layered Architecture delivers high performance for complex analytics and approvals through optimized internal boundary calls:

- **Elimination of Distributed Network Latency:** Unlike microservices, where calculating burn-up analytics requires cross-service network calls (HTTP/gRPC) across grant services, expense services, and publication services, a Layered Architecture runs in a single process memory space.
- **Database Query Aggregation:** Burn-up analytics and summary metrics ($FR - 1$) can be computed directly in the database layer using single, optimized SQL queries with indexed materialized views or aggregate sums ($SUM(grant) - SUM(expenses)$). This executes in **milliseconds**, easily meeting any sub-60-second requirement.
- **Business-Layer Caching:** Frequently accessed, slow-changing data (such as published paper indexes or overall department budget balances) can be cached directly at the Business layer using in-memory key-value stores (e.g., Redis or application memory), completely avoiding database reads on high-traffic dashboards.